Primerjava centraliziranega in distribuiranega več-agentnega iskanja poti

Andraž Šoštarič
andraz.sosteric@student.um.si
Faculty of Electrical
Engineering and Computer Science,
University of Maribor
Koroška cesta 46
SI-2000 Maribor, Slovenia

David Lipavec
david.lipavec@student.um.si
Faculty of Electrical
Engineering and Computer Science,
University of Maribor
Koroška cesta 46
SI-2000 Maribor, Slovenia

Nejc Fras
nejc.fras@student.um.si
Faculty of Electrical
Engineering and Computer Science,
University of Maribor
Koroška cesta 46
SI-2000 Maribor, Slovenia

Tadej Teršek
tadej.tersek@student.um.si
Faculty of Electrical
Engineering and Computer Science,
University of Maribor
Koroška cesta 46
SI-2000 Maribor, Slovenia

KLJUČNE BESEDE

centralizirano načrtovanje poti, distribuirano načrtovanje poti, usklajevanje agentov, izogibanje trkom, optimizacija poti

1 UVOD

V tem članku obravnavamo problem več-agentnega iskanja poti (Multi-Agent Path Finding - MAPF), ki se v praksi pojavlja v številnih domenah, kot so robotizirana skladišča, avtonomna vozila in drugi logistični sistemi.

Cilj problema je načrtovati poti za množico agentov v skupnem prostoru tako, da se izognemo medsebojnim konfliktom in hkrati optimiziramo določeno metriko, kot sta skupni čas izvajanja ali dolžina prepotovane poti.

Repozitorij z izvorno kodo projekta je na voljo na naslovu: https://github.com/Asos75/MultiAgent.

Za analizo bomo uporabili več sodobnih znanstvenih člankov, ki obravnavajo različne pristope k reševanju problema več-agentnega iskanja poti (MAPF). Kot osnovo za definicijo problema uporabljamo delo *Multi-Agent Pathfinding: Definitions, Variants, and Benchmarks* [9], ki sistematično opredeli različice problema in standardne metrike.

V nadaljevanju bomo primerjali centralizirane metode, kot sta *LaCAM: Search-Based Algorithm for Quick Multi-Agent Pathfinding* [6] in *Priority-Based Search for Multi-Agent Path Finding* [3], ki načrtujeta poti za vse agente z uporabo globalnega pogleda na problem.

Poleg tega bomo analizirali tudi decentralizirane pristope, kot sta *Follower: Learning to Solve Decentralized Lifelong Multi-Agent Pathfinding* [1] in *MATS-LP: Learning Decentralized Multi-Agent Pathfinding with Transformer-Based Policies* [8], kjer agenti sprejemajo odločitve lokalno brez centralnega nadzora.

Na ta način bomo lahko primerjali prednosti in slabosti centraliziranih in decentraliziranih pristopov z vidika učinkovitosti, skalabilnosti in kakovosti rešitev.

Kot glavni članek izberemo delo *LaCAM: Search-Based Algorithm for Quick Multi-Agent Pathfinding* [6], ki predstavlja sodoben centraliziran pristop k reševanju problema MAPF.

Izbrani članek služi kot referenčna osnova za našo analizo, saj ponuja učinkovito in skalabilno metodo globalnega načrtovanja poti za več agentov. Poleg tega je algoritem zasnovan kot iskalni postopek, kar omogoča relativno enostavno reprodukcijo eksperimentov in primerjavo z drugimi pristopi.

LaCAM bomo uporabili kot predstavnika centraliziranih metod, s katerim bomo primerjali decentralizirane pristope, kot sta Follower in MATS-LP, ter tako analizirali razlike med globalnim in lokalnim načrtovanjem poti.

Poleg analize obstoječih pristopov bomo v okviru projekta predlagali tudi lastno izboljšavo, ki uvaja koncept *lokalnih vodij* (ang. *local leaders*).

Gre za hibridni pristop med centraliziranim in decentraliziranim načrtovanjem, kjer je množica agentov razdeljena na lokalne skupine. Vsako skupino nadzira izbrani agent, lokalni vodja, ki koordinira gibanje agentov v svoji neposredni okolici in rešuje konflikte na lokalni ravni.

2 DEFINICIJA PROBLEMA

Stern et al. [9] definirajo problem več-agentnega iskanja poti (Multi-Agent Path Finding - MAPF) kot trojico

$$\langle G, s, t \rangle,$$

kjer je $G = (V, E)$ neusmerjen graf, pri čemer V predstavlja množico vozlišč, E pa množico povezav.

Podana je množica agentov $A = \{1, 2, \ldots, k\}$. Funkcija

$$s : \{1, \ldots, k\} \to V$$

preslika vsakega agenta v njegovo začetno vozlišče, funkcija

$$t : \{1, \ldots, k\} \to V$$

pa v njegovo ciljno vozlišče.

Naj bo $\pi_i[t] \in V$ položaj agenta i v diskretnem času $t \in \mathbb{N}_{\geq 0}$. Pri vsakem koraku t se agent i lahko premakne v sosednje vozlišče ali ostane na trenutnem, tj.

$$\pi_i[t+1] \in \text{neigh}(\pi_i[t]) \cup \{\pi_i[t]\},$$

kjer je $\text{neigh}(v)$ množica vozlišč, sosednjih vozlišču $v \in V$. [6]

Po Okamuri [6] se mora agent izogibati naslednjim vrstam konfliktov:

- **Vozliščni konflikt:** Do vozliščnega konflikta (ang. vertex conflict) pride, če nameravata π_i in π_j zasedeti isto vozlišče v istem trenutku, torej obstaja časovni korak t, da velja $\pi_i[t] = \pi_j[t]$. [9]
- **Zamenjalni konflikt:** Do zamenjalnega konflikta (ang. swap conflict) pride, če nameravata π_i in π_j zamenjati svoji poziciji v istem trenutku, torej obstaja časovni korak t, da velja $\pi_i[t] = \pi_j[t+1] \wedge \pi_i[t+1] = \pi_j[t]$. [9]

Rešitev MAPF za instance, omenjene zgoraj, je množica brezkonfliktnih poti $\{\pi_1, \ldots, \pi_k\}$ takih, da obstaja časovni korak $T \in \mathbb{N}_{\geq 0}$, kjer agent doseže cilj. [6]

3 SORODNA DELA

V tem razdelku izpostavimo ključna dela, relevantna za naš problem MAPF, ter njihov pomen za naš prispevek.

3.1 Centralizirani pristopi

EECBS [4] uvaja suboptimalno različico CBS z uporabo Explicit Estimation Search in dosega boljšo skalabilnost pri večjem številu agentov. Predstavlja pomembno referenco za kompromis med optimalnostjo in učinkovitostjo, ki je ključen pri primerjavi z algoritmi, kot sta LaCAM in PBS.

RHCR [5] razdeli lifelong MAPF na zaporedje krajših problemov z omejenim časovnim horizontom, kar omogoča učinkovito ponovno načrtovanje. Gre za standarden centraliziran pristop za lifelong scenarije, primerljive z našimi eksperimentalnimi nastavitvami.

3.2 Decentralizirani pristopi

PRIMAL2 [2] združuje podkrepitveno učenje in posnemanje za delovanje na podlagi lokalnih opazovanj brez centralnega koordinatorja. Predstavlja zgodnji in vpliven pristop, ki osvetljuje razvoj sodobnih decentraliziranih metod.

SCRIMP [10] uvaja komunikacijo med agenti z uporabo transformerjev, kar omogoča koordinacijo kljub omejenemu vidnemu polju. Predstavlja alternativen pristop k usklajevanju agentov v primerjavi z metodami brez eksplicitne komunikacije.

3.3 Primerjalne platforme

POGEMA [7] ponuja standardizirano platformo za evalvacijo MAPF algoritmov, vključno z generatorjem instanc in enotnimi metrikami. Omogoča pošteno in ponovljivo primerjavo različnih pristopov, kar je ključno za našo eksperimentalno analizo.

4 PREGLED OVREDNOTENJA

Za primerjavo izbranih algoritmov bomo uporabili standardne metrike s področja MAPF, ki so uveljavljene v obstoječi literaturi [7, 9]. Vrednotenje bo potekalo na dveh standardnih zbirkah testnih instanc: MovingAI MAPF benchmark [9] in POGEMA benchmark [7].

4.1 Metrike

Sum-of-Costs (SoC) je vsota časovnih korakov, ki jih vsi agenti porabijo za dosego svojih ciljnih vozlišč [9]:

$$\text{SoC} = \sum_{i=1}^{k} |\pi_i|,$$

kjer je $|\pi_i|$ dolžina poti agenta i. Nižja vrednost pomeni učinkovitejšo rešitev.

SoC je ena izmed najpogosteje uporabljenih metrik v MAPF literaturi [9], kar omogoča neposredno primerjavo z obstoječimi deli. Metrika meri skupni strošek kot vsoto dolžin poti vseh agentov in s tem neposredno zajame celostno učinkovitost sistema.

Metrika je še posebej primerna za one-shot scenarije, saj minimizacija SoC ustreza cilju minimizacije skupne porabe časa oziroma virov. Hkrati kaznuje neučinkovite poti posameznih agentov, kar spodbuja iskanje globalno učinkovitih rešitev. SoC bomo zato uporabili za vrednotenje algoritmov LaCAM in PBS na zbirki MovingAI.

Makespan je čas, ki ga porabi najdaljša pot med vsemi agenti, torej maksimum nad vsemi $|\pi_i|$ [9]:

$$\text{makespan} = \max_{i \in \{1,\ldots,k\}} |\pi_i|.$$

Metrika odraža skupno trajanje reševanja instance in je primerna za scenarije, kjer je ključen vzporedni zaključek vseh agentov. Makespan je uveljavljena metrika v MAPF literaturi [9] in je še posebej relevantna v praktičnih aplikacijah, kjer je ključen vzporedni zaključek vseh agentov.

Makespan je pri primerjavi centraliziranih algoritmov nujna dopolnitev k SoC, saj sama vsota stroškov namreč ne razkrije, ali posamezen agent morda bistveno zavlačuje celotno skupino. Z vključitvijo obeh metrik tako dobimo celovitejšo in bolj zanesljivo oceno učinkovitosti algoritma v realnih pogojih.

Stopnja uspeha (Success Rate) meri delež instanc, pri katerih vsi agenti uspešno dosežejo svoje cilje znotraj predvidenega časovnega horizonta [7].

Stopnja uspeha je primarna metrika platforme POGEMA in skupaj s pretokom (ang. throughput) tvori osnovo za vrednotenje na tej platformi [7]. Zasnovana je posebej za vrednotenje decentraliziranih metod v delno opazljivih okoljih. Metrika je za naš eksperiment ključna, saj za razliko od SoC in makespana meri tudi robustnost algoritma oziroma njegovo sposobnost, da instanco v celoti reši kljub omejenim lokalnim opazovanjem.

Čas načrtovanja merimo kot skupni čas izvajanja algoritma na posamezni instanci. Ta metrika je še posebej pomembna pri primerjavi med kategorijama, saj centralizirani načrtovalec tipično porabi več časa za globalno načrtovanje, kot decentralizirani agenti.

Čas načrtovanja je v praktičnih aplikacijah enako pomemben kot kakovost rešitve, saj algoritem z optimalno potjo, a dolgim časom načrtovanja, v realnih sistemih ni uporaben. Ta metrika nam tako omogoča primerjavo med pristopoma z vidika razširljivosti in praktične uporabnosti.

4.2 Protokol primerjave

Primerjavo bomo izvedli na dveh ravneh:

(1) **Znotraj kategorije:** LaCAM in PBS primerjamo na MovingAI zbirki po metrikah SoC, makespan in čas načrtovanja. Follower in MATS-LP primerjamo na POGEMA zbirki po metrikah throughput in stopnja uspeha.
(2) **Med kategorijama na skupni zbirki:** Obe katergoriji bomo primerjali med seboj na skupni zbirki, kjer bomo primerjali prej navedene metrike.

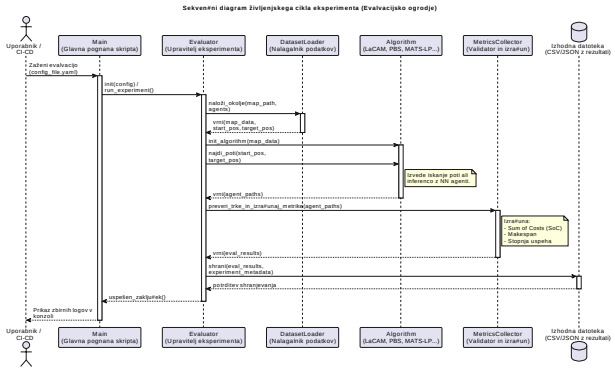
5 NAČRT REŠITVE

V tem poglavju so navedena opravila, potrebna za implementacijo projekta, ki so razdeljena v štiri razvojne iteracije (sprinte). Predstavljena sta tudi časovni razpored v obliki Ganttovega (Table 1) diagrama, ter UML diagram predlagane rešitve (Figure 1).

- **Sprint 1:**
  - Ustvarjanje repozitorija Git za nalaganje in usklajevanje programske kode.
  - Pisanje osnutka poglavja *Uvod* (predstavitev problema MAPF).
  - Pisanje osnutka poglavja *Sorodna dela* (pregled algoritmov LaCAM, PBS, Follower in MATS-LP).
  - Pregled dodatne literature ter identifikacija ključnih razlik med centraliziranimi in decentraliziranimi pristopi.
  - Definicija raziskovalnih vprašanj in začetnih hipotez.
  - Pridobivanje izvorne kode izbranih centraliziranih in decentraliziranih algoritmov.
  - Analiza strukture in zahtev posameznih implementacij.
  - Priprava eksperimentalnega okolja (namestitev knjižnic za POGEMA, orodij za C++ prevajanje in Python okolja).
- **Sprint 2:**
  - Dokončanje poglavij *Uvod* in *Sorodna dela.*
  - Pisanje poglavja *Metodologija* (definicija protokola in metrik: *SoC, makespan, uspešnost, čas*).
  - Pisanje začetnega dela poglavja *Eksperiment in analiza rezultatov.*
  - Načrt eksperimentalnega protokola (izbor instanc, parametrov, število ponovitev).
  - Reprodukcija osnovnih rezultatov avtorjev algoritmov na manjših instancah (za potrditev delovanja).
  - Implementacija lastne ideje: priprava enotnega evalvacijskega ogrodja (ang. *pipeline*) za pravično primerjavo.
  - Definicija skupnega vmesnika za integracijo različnih algoritmov.
  - Implementacija ogrodja in skript za avtomatiziran zagon eksperimentov.
  - Osnovno testiranje ogrodja in odpravljanje napak.
- **Sprint 3:**
  - Dokončanje poglavja *Metodologija.*
  - Nadaljevanje dela na poglavju *Eksperiment in analiza rezultatov.*
  - Dokončna implementacija lastnega evalvacijskega ogrodja in integracija vseh štirih algoritmov.
  - Programska implementacija in izračun izbranih metrik znotraj ogrodja.
  - Implementacija lastne izboljšave:
    * razdelitev agentov v lokalne skupine,
    * določitev lokalnih vodij,
    * implementacija lokalne koordinacije in reševanja konfliktov.
  - Integracija izboljšave v obstoječe ogrodje.
  - Validacija pravilnosti delovanja (primerjava z referenčnimi rezultati).
  - Zagon lastnega eksperimenta:
    * na izbranih problemskih instancah iz zbirk MovingAI in POGEMA,
    * pri različnih gostotah agentov in velikostih map.
- **Sprint 4:**
  - Dokončanje poglavja *Eksperimenti in analiza rezultatov.*
  - Pisanje poglavja *Zaključek* z glavno ugotovitvijo primerjave.
  - Pisanje poglavja *Povzetek.*
  - Obsežna analiza zbranih rezultatov na podlagi predlaganih metrik.
  - Primerjava centraliziranih, decentraliziranih in hibridnega pristopa.
  - Identifikacija prednosti, slabosti in omejitev posameznih pristopov.
  - Izdelava vizualizacij (grafi in tabele uspešnosti).
  - Končna revizija članka in priprava za oddajo.

Tabela 1: Ganttov diagram razvoja po sprintih (razširjena verzija).

Naloga	Tedni							
	1	2	3	4	5	6	7	8
Sprint 1								
Ustvarjanje repozitorija Git	■							
Priprava uvodnega poglavja (*Uvod*)	■	■						
Priprava poglavja (*Sorodna dela*)	■	■						
Pregled literature in identifikacija raziskovalnih vprašanj	■	■						
Pridobivanje in analiza izvorne kode algoritmov	■							
Priprava eksperimentalnega okolja	■							
Sprint 2								
Dokončanje poglavij *Uvod* in *Sorodna dela*		■	■					
Pisanje poglavja *Metodologija*			■	■				
Definicija eksperimentalnega protokola			■					
Reprodukcija osnovnih rezultatov avtorjev				■				
Razvoj enotnega evalvacijskega ogrodja (*pipeline*)				■	■			
Implementacija avtomatiziranih skript za zagon eksperimentov					■			
Sprint 3								
Dokončanje poglavja *Metodologija*					■			
Integracija vseh algoritmov v ogrodje					■			
Implementacija metrik in lokalnih vodij					■	■		
Masovni zagon eksperimentov na MovingAI/POGEMA						■	■	
Validacija rezultatov in odprava napak						■		
Sprint 4								
Dokončanje poglavja *Eksperimenti in analiza rezultatov*							■	■
Analiza grafov, metrik in primerjava pristopov							■	■
Pisanje poglavja *Zaključek*								■
Pisanje poglavja *Povzetek*								■
Končna revizija članka in priprava za oddajo								■



Slika 1: UML diagram (diagram primera uporabe / diagram razredov / sekvenčni diagram) predlagane rešitve.

LITERATURA

[1] Anton Andreychuk, Konstantin Yakovlev, Alexey Skrynnik, and Aleksandr Panov. 2024. Follower: Learning to Solve Decentralized Lifelong Multi-Agent Pathfinding. In *Proceedings of the AAAI Conference on Artificial Intelligence*. AAAI Press.

[2] Mehul Damani, Zhiyao Luo, Emerson Wenzel, and Guillaume Sartoretti. 2021. PRIMAL$_2$: Pathfinding via Reinforcement and Imitation Multi-Agent Learning — Lifelong. *IEEE Robotics and Automation Letters* 6, 2 (2021), 2666–2673. https://doi.org/10.1109/LRA.2021.3062803

[3] Jiaoyang Li, Daniel Harabor, Peter J. Stuckey, Hang Ma, and Sven Koenig. 2019. Priority-Based Search for Multi-Agent Path Finding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. AAAI Press, 181–188. https://doi.org/10.1609/aaai.v33i01.3301181

[4] Jiaoyang Li, Wheeler Ruml, and Sven Koenig. 2021. EECBS: A Bounded-Suboptimal Search for Multi-Agent Path Finding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35. 12353–12362. https://doi.org/10.1609/aaai.v35i14.17341

[5] Jiaoyang Li, Andrew Tinka, Scott Kiesel, Joseph W. Durham, T. K. Satish Kumar, and Sven Koenig. 2021. Lifelong Multi-Agent Path Finding in Large-Scale Warehouses. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35. 11272–11281. https://doi.org/10.1609/aaai.v35i13.17344

[6] Keisuke Okumura. 2023. LaCAM: Search-Based Algorithm for Quick Multi-Agent Pathfinding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. AAAI Press, 11655–11662. https://doi.org/10.1609/aaai.v37i10.26377

[7] Alexey Skrynnik, Anton Andreychuk, Anatolii Borzilov, Alexander Chernyavskiy, Konstantin Yakovlev, and Aleksandr Panov. 2024. POGEMA: A Benchmark Platform for Cooperative Multi-Agent Navigation. arXiv:2407.14931 [cs.LG] https://arxiv.org/abs/2407.14931

[8] Alexey Skrynnik, Anton Andreychuk, Konstantin Yakovlev, and Aleksandr Panov. 2024. Learning Decentralized Policies for Multi-Agent Pathfinding with Transformers. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

[9] Roni Stern, Nathan Sturtevant, Ariel Felner, Sven Koenig, Hang Ma, Thayne Walker, Jiaoyang Li, Dor Atzmon, Liron Cohen, T. K. Satish Kumar, Eli Boyarski, and Roman Bartak. 2019. Multi-Agent Pathfinding: Definitions, Variants, and Benchmarks. In *Proceedings of the 12th Annual Symposium on Combinatorial Search (SoCS)*.

[10] Yutong Wang, Bairan Xiang, Shao Huang, and Guillaume Sartoretti. 2023. SCRIMP: Scalable Communication for Reinforcement- and Imitation-Learning-Based Multi-Agent Pathfinding. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. https://doi.org/10.1109/IROS55552.2023.10341379